

CIMA

Control Interno y Monitoreo de Adultos Mayores

Especificación técnica

27 de septiembre de 2026

Índice

1. Especificación técnica	1
1.1. Historial	1
1.2. 1. Propósito	1
1.3. 2. Estado verificado	2
1.4. 3. Stack y ejecución	3
1.5. 4. Autenticación y sesiones	3
1.6. 5. Roles y permisos	3
1.7. 6. API implementada	4
1.8. 7. Validación y pruebas existentes	5
1.9. 8. Seguridad y privacidad	5
1.10. 9. Pendientes funcionales	5
1.11. 10. Siguiendo incremento recomendado	6
1.12. 11. Criterios para cerrar un módulo	6

1. Especificación técnica

Versión del documento: 0.5

Fecha de corte: 27 de septiembre de 2026

Estado: CRUD y ficha de residentes, autenticación, RBAC inicial, dashboard base y habitaciones con asignación e historial implementados. Estadías y otros módulos de negocio siguen pendientes.

Versión publicada: v0.3 continúa siendo la última versión con release/tag; v0.5 describe el estado del árbol de trabajo y no implica un release publicado.

1.1. Historial

- **0.3:** ficha personal, contactos y validaciones de datos.
- **0.4:** CRUD de residentes, ficha editable, archivado lógico y timestamps.
- **0.5:** login JWT, renovación de sesión, usuarios/roles, RBAC por endpoint, dashboard CRM inicial, habitaciones y asignaciones trazables, mensajes API en español, locale **es-CL**, UTF-8, fecha chilena con autoformato y calendario; integración API/MongoDB aislada y actualización del estado real de módulos y riesgos.

1.2. 1. Propósito

CIMA es una aplicación web interna e independiente para apoyar la operación de una residencia de personas mayores en Chile. Centralizará progresivamente la gestión de residentes, habitaciones, alimentación, atención clínica, medicamentos, estadías, cargos, pagos, documentos y auditoría.

El frontend no se conecta directamente a MongoDB. El backend es la autoridad para validar datos y autorizar operaciones.

Navegador React/TypeScript

| HTTPS en producción; HTTP local en desarrollo

v
API Go + Chi
|
v
MongoDB

1.3. 2. Estado verificado

Área	Implementado actualmente	Límites conocidos
Entorno	Docker Compose con MongoDB 7, API Go y frontend React/Vite. Node, Go y MongoDB se ejecutan en contenedores.	El host solo requiere Docker Desktop y Git para el flujo normal.
Residentes	Crear, listar, consultar por ID, actualizar identidad, archivar lógicamente; timestamps <code>createdAt</code> , <code>updatedAt</code> , <code>archivedAt</code> .	Sin búsqueda, filtros, paginación ni restauración.
Habitaciones	Catálogo, código único, capacidad, estado, asignación/liberación de residentes activos, ocupación atómica e historial con actor/hora; API y UI protegidas por RBAC.	Sin módulo de mantención ni baja de habitaciones.
Ficha	Fecha de nacimiento, teléfono, correo, contacto principal y de emergencia; lectura y actualización parcial.	Se almacena como subdocumento de residente. No incluye antecedentes médicos ni documentos.
Fechas e idioma	HTML <code>lang="es-CL"</code> , UTF-8; presentación <code>dd/mm/aaaa</code> , autoformato, calendario nativo y validación de fecha; API conserva <code>AAAA-MM-DD</code> .	La localización visual del calendario nativo depende del navegador/SO.
Autenticación	Login, access token JWT de 15 minutos, refresh JWT de 7 días y perfil <code>/auth/me</code> . Contraseñas con bcrypt.	Refresh no es revocable; no hay logout de servidor ni gestión de sesiones.
Usuarios	API para listar, consultar, crear y actualizar usuarios, roles, permisos y contraseña.	No hay pantalla de administración de usuarios ni baja de usuarios.
RBAC	Middleware <code>requirePermission</code> sobre rutas de negocio; 401 sin token/inválido, 403 sin permiso; rol <code>admin</code> omite la comprobación granular.	La UI solo adapta navegación; la API sigue siendo el control de seguridad. No hay autorización por recurso/tenant ni revocación inmediata de access tokens.
Dashboard	Login, contexto de usuario/rol, módulos filtrados por permiso y métricas de residentes disponibles.	<code>/dashboard</code> solo entrega estado; no es aún un dashboard operacional completo.
Módulos restantes	Rutas protegidas para atención clínica, medicamentos, facturación y documentos; auditoría tiene GET protegido.	Excepto el catálogo de habitaciones, siguen como placeholders; los POST responden HTTP 501.
Mensajes	Mensajes de error y estados expuestos por la API están en español.	Los valores de enumeraciones internas/API como <code>active</code> , <code>inactive</code> y nombres de campos siguen siendo identificadores técnicos.

1.4. 3. Stack y ejecución

- **API:** Go 1.23, Chi, MongoDB Go Driver, JWT HS256 y bcrypt.
- **Frontend:** React 18, TypeScript, Vite y Vitest.
- **Datos:** MongoDB 7; colecciones activas **residents**, **users** y **rooms**.
- **Ejecución:** Docker Compose; pruebas y builds se ejecutan dentro de contenedores.
- **Servicios locales:** frontend <http://localhost:5173>, API <http://localhost:8080>, health GET /health, MongoDB localhost:27017.
- **Base de datos:** nombre por defecto cima; colecciones activas **residents**, **users** y **rooms**.

Comandos habituales:

```
docker compose up --build
docker compose ps
docker compose logs -f backend
```

```
docker build --target builder -t cima-backend-check ./backend
docker run --rm cima-backend-check go test ./...
```

```
docker build --target build -t cima-frontend-check ./frontend
docker run --rm cima-frontend-check npm test -- --run
```

```
docker compose --profile integration run --rm integration-tests
```

No ejecutar `npm install`, Go ni MongoDB en el host como parte del flujo previsto.

1.5. 4. Autenticación y sesiones

Rutas públicas/autenticadas:

```
POST /auth/login    público; responde access token, refresh token y usuario
POST /auth/refresh  público con refresh token en JSON
GET  /auth/me       requiere bearer token válido
```

El access token dura 15 minutos; el refresh token dura 7 días. El frontend guarda ambos en `sessionStorage`, consulta `/auth/me` al iniciar y renueva el access token ante un 401. Al fallar la renovación, limpia la sesión y los datos protegidos en memoria.

El usuario inicial de desarrollo se crea como **admin** con contraseña **admin123** si no existe. Esta credencial y el secreto JWT de fallback son solo para desarrollo y deben cambiarse antes de cualquier despliegue. Los usuarios existentes no se migran ni se reconfiguran automáticamente al volver a iniciar la API.

1.6. 5. Roles y permisos

Permisos definidos en backend:

<code>resident.read</code>	<code>resident.create</code>	<code>resident.update</code>	<code>resident.delete</code>
<code>user.read</code>	<code>user.write</code>		
<code>room.read</code>	<code>room.write</code>		
<code>medical.read</code>	<code>medical.write</code>		
<code>medication.read</code>	<code>medication.write</code>		
<code>billing.read</code>	<code>billing.write</code>		
<code>document.read</code>	<code>document.write</code>		
<code>audit.read</code>	<code>dashboard.read</code>		
<code>diet.read</code>	<code>food.update</code>		

Resumen de permisos base por rol:

Rol	Acceso principal
admin	Bypass de middleware; catálogo administrativo completo.
manager	Residentes, lectura de usuarios, habitaciones, lectura clínica, dashboard y auditoría.
nurse	Lectura/actualización de residentes, atención clínica y medicamentos.
kitchen	Lectura de residentes, lectura de dietas y actualización de alimentación.
reception	Lectura/creación/actualización de residentes y habitaciones.
accounting	Lectura de residentes, facturación y dashboard.

El catálogo es el estado del código actual; no implica que los módulos `diet` y `food` tengan endpoints implementados. La interfaz oculta módulos sin permiso, pero esto no sustituye el middleware de backend.

1.7. 6. API implementada

Base de negocio: `/api/v1`. Salvo `health` y `auth`, las rutas listadas requieren bearer token y el permiso indicado.

Método y ruta	Permiso	Estado
GET <code>/health</code>	Público	Implementado.
POST <code>/auth/login</code>	Público	Implementado.
POST <code>/auth/refresh</code>	Refresh token	Implementado.
GET <code>/auth/me</code>	Bearer token	Implementado.
GET <code>/api/v1/users/</code>	<code>user.read</code>	Implementado.
GET <code>/api/v1/users/{id}</code>	<code>user.read</code>	Implementado.
POST <code>/api/v1/users/</code>	<code>user.write</code>	Implementado.
PATCH <code>/api/v1/users/{id}</code>	<code>user.write</code>	Implementado.
GET <code>/api/v1/residents/</code>	<code>resident.read</code>	Implementado.
GET <code>/api/v1/residents/{id}</code>	<code>resident.read</code>	Implementado.
POST <code>/api/v1/residents/</code>	<code>resident.create</code>	Implementado.
PATCH <code>/api/v1/residents/{id}</code>	<code>resident.update</code>	Implementado.
DELETE <code>/api/v1/residents/{id}</code>	<code>resident.delete</code>	Archivado lógico.
GET <code>/api/v1/residents/{id}/profile/</code>	<code>resident.read</code>	Implementado.
PATCH <code>/api/v1/residents/{id}/profile/</code>	<code>resident.update</code>	Implementado.
GET <code>/api/v1/rooms/</code>	<code>room.read</code>	Catálogo implementado.
GET <code>/api/v1/rooms/{id}</code>	<code>room.read</code>	Implementado.
POST <code>/api/v1/rooms/</code>	<code>room.write</code>	Implementado; código único y capacidad positiva.
PATCH <code>/api/v1/rooms/{id}</code>	<code>room.write</code>	Implementado; actualización parcial.
POST <code>/api/v1/rooms/{id}/assign</code>	<code>room.write + resident.read</code>	Implementado; residente activo, capacidad y duplicidad verificadas.
POST <code>/api/v1/rooms/{id}/release</code>	<code>room.write + resident.read</code>	Implementado; registra liberación en historial.
GET <code>/api/v1/rooms/{id}/history</code>	<code>room.read</code>	Historial de asignación/liberación.
GET <code>/api/v1/dashboard/</code>	<code>dashboard.read</code>	Stub de estado.
GET <code>/api/v1/audit/</code>	<code>audit.read</code>	Stub de estado; no consulta auditoría.
GET/POST <code>/api/v1/documents/</code>	<code>document.read / document.write</code>	GET stub; POST HTTP 501.
GET/POST <code>/api/v1/billing/</code>	<code>billing.read / billing.write</code>	GET stub; POST HTTP 501.

Método y ruta	Permiso	Estado
GET/POST /api/v1/medical/	<code>medical.read</code> / <code>medical.write</code>	GET stub; POST HTTP 501.
GET/POST /api/v1/medication-events/	<code>medication.read</code> / <code>medication.write</code>	GET stub; POST HTTP 501.

Las solicitudes JSON tienen límite de 1 MiB y rechazan campos desconocidos en los DTOs que usan `decodeStrictJSON`. CORS permite el origen configurado, métodos requeridos y encabezados `Authorization` y `Content-Type`.

1.8. 7. Validación y pruebas existentes

- Backend: validación de estado del residente, JSON estricto, bcrypt, emisión/lectura JWT, roles/permisos y middleware RBAC.
- RBAC unitario: falta token → 401; token inválido → 401; permiso ausente → 403; permiso presente → 200; admin sin permisos explícitos → bypass.
- API ejecutada: los nueve GET protegidos de usuarios, dashboard, auditoría, documentos, facturación, habitaciones, atención clínica, eventos de medicamentos y residentes devolvieron 401 sin token y 200 con admin.
- Integración de habitaciones: alta, código duplicado, lectura, edición y capacidad inválida probados contra MongoDB aislado; enfermería recibe 403 por falta de `room.read`. Asignación/liberación se implementaron después de esa ejecución y quedan pendientes de validación automatizada.
- Frontend: pruebas Vitest para permisos y formato/validación de fechas; total actual 7 casos.
- Verificaciones: build frontend/backend en Docker, suite Go, smoke HTTP y `git diff --check`.

Pendiente: pruebas E2E Playwright mantenibles; automatizar preflight CORS como test y ampliar la integración conforme se implementen nuevos módulos. El perfil de pruebas no utiliza la base de desarrollo **cima**.

1.9. 8. Seguridad y privacidad

Controles actuales: contraseñas bcrypt, JWT HS256, validación backend, JSON estricto, límite de body, CORS por origen, permisos por ruta, errores de API en español y no exposición del hash de contraseña en JSON.

Riesgos/pendientes antes de datos reales o producción:

- Reemplazar secreto JWT de fallback y credenciales seed; exigir configuración fuerte por ambiente y permitir rotación.
- Migrar de bcrypt a Argon2id si se mantiene ese requisito de política.
- Distinguir criptográficamente access/refresh tokens; persistir/revocar sesiones, agregar logout y limitar reutilización del refresh token.
- Resolver escalamiento de privilegios en administración delegada de roles/permisos; restringir quién puede asignar `admin` o permisos superiores.
- Reducir la vigencia efectiva de permisos modificados: claims de permisos en access tokens pueden quedar vigentes hasta su expiración.
- Rate limiting, HTTPS, cabeceras de seguridad, monitoreo y pruebas de recuperación de respaldos.
- Auditoría real para acceso y cambios sensibles; nunca registrar contraseñas, tokens ni datos médicos innecesarios.
- Revisión legal y de privacidad aplicable en Chile, retención y respuesta a incidentes.

No usar información real de residentes ni desplegar en producción con la configuración actual.

1.10. 9. Pendientes funcionales

1. Implementar estadías y asociar sus transiciones con asignaciones de habitaciones.
2. Implementar alimentación como módulo separado y limitar la vista de cocina a la información necesaria.
3. Implementar atención clínica y medicamentos/eventos con trazabilidad inmutable de administración.
4. Implementar auditoría real, protegida y con minimización de datos.
5. Completar dashboard con métricas reales y vistas según rol; agregar interfaz de administración de usuarios.
6. Diseñar y construir documentos privados, cargos/pagos y sus permisos.
7. Automatizar E2E, accesibilidad WCAG 2.2 AA, CI y hardening previo a producción.

1.11. 10. Siguiente incremento recomendado

Siguiente incremento: implementar estadías con ingreso, egreso, estado e historial; vincularlas a la asignación actual de habitación. La suite de integración se ejecuta con `docker compose --profile integration run --rm integration-tests` y usa `cima_integration`, sin depender de datos locales.

1.12. 11. Criterios para cerrar un módulo

Cada módulo requiere modelo/índices, DTOs y validación, autorización backend y por recurso cuando corresponda, endpoints reales, interfaz con carga/vacío/error/éxito, auditoría según sensibilidad, pruebas unitarias e integración, documentación y revisión de accesibilidad. Todo debe compilar/probar en Docker sin dependencias locales.